TenderSense — Build Plan (BRAC IT CodeSprint, demo 12 Sep 2026)

Context

BracIT bids for government and development-sector contracts but finds out about tenders too late. The BRD (author: Aisha Tanjina, PIN 2348) proposes **TenderSense**: a daily pipeline that collects tenders from e-GP Bangladesh and World Bank, matches them semantically against BracIT's capability profile, applies rules-based eligibility filtering, and delivers a ranked S/A/B/C shortlist.

We are building someone else's submitted idea, and the BRD budgets 7 days — we have **4** (8–11 Sep, demo 12 Sep). This plan is sized to that, with above-BRD features chosen specifically to win the "AI readiness" framing rather than to add surface area.

Reconnaissance already completed (8 Sep 2026) — this de-risks the plan substantially:

- **e-GP is scrapable.** Live tenders pulled today. No login, no captcha on browsing (captcha exists only on the login form). ~3,270 live tenders ("Page 1 of 327" × 10).
- **The listing needs a real browser.** `AllTenders.jsp` ships an empty `<table id="resultTable">`; jQuery POSTs to `/TenderDetailsServlet` to fill it. Plain `curl` against that servlet returns `No records found` for every parameter combination tried (viewType Live/Archive/empty, all `procMethod`, all `isFrame`). Headless Chrome renders it correctly first try.
- **Detail pages need no browser.** `POST /resources/common/ViewTender.jsp` with `id=<tenderId>&h=t` returns Ministry · Division · Organization · Procuring Entity + code + district · Procurement Nature/Type/Method · Budget Type · Source of Funds · Document Price · Publishing & Closing datetimes · Brief Description · **Eligibility of Tenderer** (free text). That last field feeds the rules engine directly.
- **e-GP RSS is dead.** `/RSS/DailyTender.xml`, `/RSS/WeeklyTender.xml`, `/RSS/RSS_eGP.xml` all 404.
- **World Bank needs no scraper.** `https://search.worldbank.org/api/v2/procnotices?format=json` — 417,948 notices, no auth. Confirmed filters: `project_ctry_name_exact=Bangladesh` (8,927), `notice_type_exact=Invitation for Bids` (45,366), `qterm=<free text>`. ⚠️ `countryname_exact` and `cntry_exact` **silently return the unfiltered total** — use `project_ctry_name_exact` or the filter will appear to work while doing nothing.

Team: 3 fullstack developers. **Assets:** BracIT capability profile + 40 labeled tenders already in hand. **AI budget:** free only.

Tech decisions

Concern	Decision	Why
Runtime	Spring Boot 4.1.1, **Java 17 toolchain**	Only JDK 17 and 8 are installed. `build.gradle` currently asks for 25 — Gradle would have to provision it. Change to 17 first.
DB	PostgreSQL 16 + **pgvector** via Docker	Postgres 16 and a working Docker daemon are already on the machine; `pgvector/pgvector` image confirmed available.
Embeddings	**`spring-ai-starter-model-transformers` 2.0.1** (local ONNX, all-MiniLM-L6-v2, 384-dim)	**Zero API key, zero cost, runs offline in-JVM.** This is literally the "sentence-transformers fallback" the BRD names — and it makes the demo immune to venue network failure.

Concern	Decision	Why
Vector store	`spring-ai-starter-vector-store-pgvector` 2.0.1	Verified on Maven Central.
Summaries	Free LLM behind an interface, **template fallback**	BRD states summaries are never used for pass/fail, so a dead API degrades text quality only, never correctness.
e-GP listing	Selenium 4 + headless Chrome 144	Only way to get the grid. Selenium Manager auto-fetches the driver.
e-GP detail	Java `HttpClient` + Jsoup	Plain POST — no browser, parallelisable.
World Bank	`RestClient` + Jackson	It's a JSON API.
Frontend	**Angular 21.2.x** (team decision)	Separate `frontend/` project consuming a REST API.
Node	**v20.19.6 via `nvm use`**	⚠️ Active Node is **v20.12.2**, which satisfies *no* current Angular. v20.19.6 is already installed via nvm — zero download. Angular 22 would need Node 22.22+/24.15+; the installed v25.1.0 does **not** qualify (Angular excludes odd-numbered Node).
API style	REST + DTOs, `@RestController` only	No view layer in Spring at all.
Serving	Dev: `proxy.conf.json` → :8080. Demo: `ng build` → `src/main/resources/static/`	Dev proxy and a single-origin demo jar mean **CORS is never live in the demo path**.

Free AI keys (for summaries only — embeddings need none)

All three endpoints were probed today and are reachable:

- **Google AI Studio** — `aistudio.google.com/apikey`. Free tier, `gemini-2.5-flash`. **First choice.**
- **Groq** — `console.groq.com`. Free tier, very fast.
- **OpenRouter** — `openrouter.ai/keys`. 19 models with `:free` pricing confirmed today.

Someone on the team must create the key manually. If none is obtained, the template summary writer carries the demo.

Architecture

Layered packages per team convention, all under `com.bracit.tendersense`.

```
entity/          JPA entities + enums
repository/      Spring Data JPA interfaces
dto/             API request/response records — entities are NEVER serialised to Angular
service/         interfaces only
service/impl/    implementations
controller/      @RestController only — no view layer anywhere in Spring
config/          CorsConfig · SchedulingConfig · SeleniumConfig · EmbeddingConfig · OpenApiConfig
scheduler/       the six scheduled jobs
util/            EgpHtmlParser · WorldBankJsonMapper · TextChunker · CosineSimilarity · DhakaTime
exception/       GlobalExceptionHandler · ScrapeException · ProfileNotFoundException
```

`entity/` — `Tender`, `CapabilityProfile`, `PastProject`, `Certification`, `MatchResult`, `EligibilityVerdict`, `EligibilityGap`, `BidDecision`, `PipelineRun`, `RawSnapshot`. Enums: `SourcePortal`, `MatchGrade` (S/A/B/C),

`BidAction` (BID/HOLD/SKIP), `EligibilityStatus`.

`service/` interfaces → `service/impl/` — this is where the convention pays off, because the three swap-points the demo depends on land naturally on interface + multiple impls:

Interface	Implementations	Why multiple
`TenderFetchService`	`EgpTenderFetchServiceImpl` · `WorldBankTenderFetchServiceImpl` · `CachedTenderFetchServiceImpl`	**Demo safety net.** Profile-selected; if venue wifi dies, cached replay is a flag, not a code change
`MatchingService`	`EmbeddingMatchingServiceImpl` · `KeywordMatchingServiceImpl`	**The benchmark.** Semantic vs keyword becomes injecting a different bean, not a parallel codebase
`SummaryService`	`LlmSummaryServiceImpl` · `TemplateSummaryServiceImpl`	**No-key fallback.** `@ConditionalOnProperty` on the API key; template impl is `@Primary` until a key exists

Remaining services, one impl each: `TenderIngestionService`, `EligibilityService`, `GradeCalibrationService`, `EvaluationService`, `DigestService`, `FeedbackService`.

> Naming note: the ingestion abstraction is `TenderFetchService`, **not** `TenderSource` — the latter would collide with the `SourcePortal` concept in `entity/`. Worth fixing now, not on Day 3.

`controller/` — `TenderController`, `BenchmarkController`, `PipelineController`, `ProfileController`, `FeedbackController`.

API contract — freeze this on Day 1

Track C cannot build Angular against an API that is still moving. These endpoints get agreed and stubbed with fixture data on Day 1 so frontend and backend proceed in parallel from hour one.

```
GET  /api/tenders?grade=&eligible=&source=&page=&size=   → Page<TenderSummaryResponse>
GET  /api/tenders/{id}                                   → TenderDetailResponse
GET  /api/tenders/{id}/evidence                          → MatchEvidenceResponse   (feature 2)
GET  /api/tenders/{id}/eligibility                       → EligibilityGapResponse  (feature 3)
POST /api/tenders/{id}/decision                          → BidDecisionRequest      (feature 4)
GET  /api/benchmark                                      → BenchmarkResponse       (feature 1)
GET  /api/profile          PUT /api/profile              → CapabilityProfileDto
POST /api/pipeline/run     GET /api/pipeline/runs        → PipelineRunResponse     (features 5,6)
```

Angular app

```
frontend/src/app/
├── core/       ApiService · models/ · http interceptors
├── features/   shortlist/ · tender-detail/ · benchmark/ · profile/ · pipeline/
└── shared/     grade-badge · urgency-chip · evidence-panel · eligibility-gap-list
```

Standalone components, typed `HttpClient`, signals for state. **No component library** — Angular Material's theming will eat half a day you do not have; hand-rolled CSS on five screens is faster.

`shortlist/` is the demo screen and gets built first. `benchmark/` is the second priority — it is feature 1, the thing that wins the "beats keyword search" criterion on stage.

Scheduling

The BRD says "pulls new tenders automatically each day" and "delivered every morning". A single daily job would satisfy the letter of that and be the wrong design, because the two e-GP operations have very different costs:

- **Discovery** (listing page 1, newest first) — cheap, seconds, finds what's new.
- **Full crawl** (327 listing pages + up to 3,270 detail POSTs at 1 req/sec) — **~1 hour**.

So the pipeline is split into five jobs at different cadences rather than one nightly monolith. All cron expressions are Spring 6-field, pinned to `Asia/Dhaka` — never server-local, or the "every morning" digest drifts.

Job	Cadence	Cron (`zone = "Asia/Dhaka"`)	Cost
`egpDiscovery`	Every 30 min, 08:00–20:30	`0 0/30 8-20 * * *`	Seconds — 1–3 listing pages
`egpDetailDrain`	Continuous queue worker	fixed delay 1s, rate-limited	1 req/sec, only new IDs
`egpReconcile`	Nightly 02:00	`0 0 2 * * *`	~1 hour, full 327-page crawl
`worldBankSync`	Every 6 hours	`0 15 */6 * * *`	Seconds — it's a JSON API
`urgencyRefresh`	Daily 05:30	`0 30 5 * * *`	Seconds — DB only
`morningDigest`	Daily 08:00	`0 0 8 * * *`	Seconds — the BRD deliverable

Minutes are staggered (`:00/:30`, `:15`, `:30`) so jobs never collide on the single Selenium instance.

Why 30 minutes for discovery, not 5 and not 24 hours. Observed e-GP closing windows are ~12 days (tenders published 08-Sep close 20-Sep), so minute-level polling buys nothing real. But the BRD's promise is "before competitors notice", and discovery is nearly free — so 30 minutes during Bangladesh business hours is the honest balance. Bangladesh's government weekend is **Friday–Saturday**, so yield those days is near zero; the job still runs, because the cost is trivial and it catches off-cycle publications.

How discovery stays cheap. The listing is sorted newest-first (confirmed: today's tenders sat on page 1). Discovery walks pages only until it has seen **20 consecutive already-known tenderIds** (two full pages), then stops — typically 1–3 pages. New IDs go on a queue; `egpDetailDrain` fetches their detail pages at 1 req/sec. This is why the expensive detail POST never runs on a schedule at all: it is demand-driven off discovery.

Why a nightly full reconcile is still needed. Discovery only sees *new* tenders. The reconcile pass catches what it structurally cannot: corrigenda and amendments to tenders already ingested, status transitions (Live → Closed), and anything missed during a portal outage. It also refreshes the snapshot corpus.

Scoring is event-driven, not scheduled. A tender is embedded and scored on ingest. A full re-score is triggered only by a capability-profile edit or an embedding-model version change — both recorded in the provenance trail (feature 5).

Operational requirements.

- **Non-reentrant.** A slow reconcile must never overlap the next discovery. Single instance means a simple in-process lock suffices; note the constraint explicitly so nobody scales to two pods and silently double-crawls the portal.
- **Backoff.** On repeated 5xx or a Selenium failure, back off exponentially and stop for the cycle. Do not hammer a government portal.
- **Config, not code.** Every cadence lives in `application.properties` as `tendersense.schedule.*`, so it can be changed — or demonstrated being changed — without a rebuild.
- `demo` **profile disables all schedulers.** Only the manual "Run now" trigger fires. A reconcile kicking off mid-presentation and locking the DB is an avoidable way to lose.

Features beyond the BRD (chosen to win, not to pad)

Ranked by judge impact per hour spent.

1. **Live benchmark page — semantic vs keyword, on stage.** The BRD's headline success criterion is "beats keyword search on top-5". Most teams will *assert* this. We will show precision@5 for both systems computed live on the held-out tenders. **This requires building the BM25/full-text baseline as a real deliverable** — it is not rhetorical.
2. **Evidence-backed explanations.** For each match, surface which profile capability matched which tender sentence, with per-chunk cosine scores. Converts "the AI said 0.87" into visible reasoning. Directly answers the "is this AI trustworthy" question judges will ask.
3. **Eligibility gap report.** Don't just fail a tender — say *why and by how much*: "fails turnover by BDT 40M", "would qualify with ISO 27001". The BRD asks for "what's needed to bid"; this makes it quantitative and actionable.
4. **Bid/no-bid decision log with feedback loop.** BRD defers this to "future". Implementing it — user marks a tender irrelevant, the ranking adapts via negative-example centroid adjustment — demonstrates a system that *learns*, which is the difference between "we called an API" and "we built an AI system".
5. **Full provenance / audit trail.** Every score traceable: raw HTML snapshot → parsed fields → embedding model version → score → grade → threshold rationale. The BRD's own footer reads *CMMI Dev/3 · ISO 27001 · ISO 9001* — an auditable pipeline speaks that organisation's language and is what enterprise "AI readiness" actually means.
6. **Run telemetry.** Per-tender processing time displayed live, proving the <60s criterion on stage instead of claiming it.
7. **Corrigendum/duplicate detection.** e-GP republishes amendments; dedupe on reference no + PE code so the shortlist doesn't show the same tender three times. Cheap, and its absence is visible.
8. **Stretch — Bangla matching.** Swap the ONNX model for `paraphrase-multilingual-MiniLM-L12-v2` so Bangla tender text matches an English profile. BRD defers this. High differentiation for a Bangladesh-market tool. **Attempt only if Day 3 runs ahead of schedule** — it is a model swap plus revalidation, and a failed swap must not touch the working English path.

Day-by-day

Track owners: **A** = ingestion + scheduling · **B** = AI matching + eval · **C** = entity/repository/dto/controller + **Angular**. All three are fullstack, so the split is by ownership, not capability. C owns the API contract because C consumes it — that keeps the contract honest and removes a handoff.

Day 0 — today, 8 Sep (remaining hours)

Highest-value action available, and it is time-sensitive.

- **[A] Snapshot the live e-GP corpus now.** ~3,270 live tenders, listing via headless Chrome, then detail POSTs at ~1 req/sec (≈1 hour). Write raw HTML to disk keyed by tenderId. **This is insurance: once on disk, no portal change, block, or outage between now and 12 Sep can hurt us.**
- **[A]** Fix `build.gradle` : Java toolchain 25 → 17. Run `./gradlew build` to confirm. If Boot 4.1 rejects 17, `sudo apt install openjdk-21-jdk` and set 21.
- **[B]** Load the capability profile and 40 labeled tenders. Split **28 dev / 12 held-out** and commit the split. Nobody looks at the 12 until Day 3 — that is what makes the benchmark credible.
- **[C]** Docker compose for `pgvector/pgvector` ; confirm connection from Spring.
- **[C]** `nvm use 20.19.6` (pin it in `frontend/.nvmrc` so nobody lands on v20.12.2 and loses an hour to a cryptic engine error), `ng new frontend --standalone --style=css --routing`, and a `proxy.conf.json` pointing `/api` at :8080.

Day 1 — 9 Sep · Ingestion + persistence

- **[A]** `TenderSource` interface; `EgpBangladeshSource` (Selenium listing + Jsoup detail); `CachedFileSource` replaying Day-0 snapshots. Parse on label text (`Ministry :`, `Eligibility of Tenderer :`), **never positional index** — the table shifts between tender types.
- **[A]** `WorldBankSource` via RestClient. Use `project_ctry_name_exact`; add an assertion that a filtered count is strictly less than the unfiltered total, so a silently-ignored filter fails loudly.
- **[C]** `entity/` + `repository/` + schema; corrigendum dedupe on reference no + PE code.
- **[C] Freeze the API contract** (endpoints above) and ship all eight controllers returning hardcoded fixture DTOs. This is the day's most important unblocking act: Angular then develops against a real API from Day 1 instead of waiting on Day 3 integration.
- **[B]** Wire `spring-ai-starter-model-transformers`; embed the capability profile; verify 384-dim vectors land in pgvector.

Gate: live tender fetched → parsed → persisted → embedded, end to end.

Day 2 — 10 Sep · Matching, eligibility, explanations

- **[B]** `EmbeddingMatcher` — chunk tender text and profile, cosine, aggregate to a score.
- **[B]** `KeywordBaselineMatcher` — Postgres full-text/BM25. **Deliverable, not a strawman;** build it honestly or the benchmark proves nothing.
- **[B]** `GradeCalibrator` — derive S/A/B/C cut points from the **28 dev tenders** by maximising separation between labeled-relevant and labeled-irrelevant. Record the derivation. "We set thresholds where the labeled sets separate" is defensible; "0.8 felt right" is not.
- **[A]** `RuleEngine` + turnover/certification/geography rules, parsing the e-GP *Eligibility of Tenderer* text. Plus `EligibilityGapReport` (feature 3). **Rules stay rules — no LLM anywhere in the pass/fail path.** This separation is the BRD's strongest design decision and must be explicit in the demo.
- **[B]** `SummaryService` + `TemplateSummaryServiceImpl` first, `LlmSummaryServiceImpl` second, so a missing key never blocks anyone. `EvidenceExtractor` in `util/` (feature 2).
- **[C]** Angular `shortlist/` screen against live data — the demo screen, built first. Then `tender-detail/` with the evidence panel and eligibility gaps.

Gate: all 28 dev tenders scored, graded, eligibility-checked, with explanations.

Day 3 — 11 Sep · Delivery, benchmark, hardening

- **[C]** Angular `benchmark/` screen (feature 1) — **first and only run against the held-out 12**, precision@5 semantic vs keyword, side by side. Then `pipeline/` (run trigger + telemetry) and `profile/`.
- **[C] Production build wired**: Gradle task runs `ng build` and copies `frontend/dist/` into `src/main/resources/static/`, so the demo is one `bootRun` on one origin with no proxy and no CORS. **Do this by Day 3 midday, not demo morning** — it is the classic Angular-plus-Spring trap, and the failure mode (blank page, 404 on deep links) always shows up first in the packaged build.
- **[B]** `DecisionLog` + `FeedbackLoop` (feature 4).
- **[A]** The five scheduled jobs per **Scheduling** above (`egpDiscovery`, `egpDetailDrain`, `egpReconcile`, `worldBankSync`, `urgencyRefresh`, `morningDigest`), cadences externalised to `tendersense.schedule.*`, plus the manual "Run now" trigger and the `demo` profile that disables all of them; `RunTelemetry` (feature 6); provenance view (feature 5).
- **[A]** Verify the `CachedFileSource` profile switch works cold.
- **All:** freeze code by evening. **Stretch only if ahead: Bangla model swap (feature 8), on a branch, never on main.**

Day 4 — 12 Sep · Demo

- Morning: two full dress rehearsals — once live, once forced onto `CachedFileSource`.
- The three BRD demo cases (S/eligible, A/eligible, S/turnover-flag) plus one live e-GP pull on stage.
- Disclose the backup exists, as the BRD instructs.

Files to create or change

Change: `build.gradle` — Java toolchain 25 → 17; add Spring AI 2.0.1 BOM, `spring-ai-starter-model-transformers`, `spring-ai-starter-vector-store-pgvector`, Selenium 4, Jsoup; **no Thymeleaf**; add the `ngBuild` task that copies `frontend/dist/` into `src/main/resources/static/`.
`src/main/resources/application.properties` — datasource, pgvector, `tendersense.schedule.*`, profiles (`live` / `cached` / `demo`).

Create (backend): `docker-compose.yml` (pgvector); the nine packages above; the labeled dataset and capability profile under `src/main/resources/data/`; snapshots under `data/snapshots/` (**gitignored — this will be a few hundred MB**).

Create (frontend): `frontend/` Angular project, `frontend/.nvmrc` pinning `20.19.6`, `frontend/proxy.conf.json`, plus `core/`, `features/`, `shared/` as laid out above.

The existing `TenderSenseApplication.java` needs no change beyond `@EnableScheduling`.

Verification

Environment

```
./gradlew build                      # confirms Java toolchain choice
docker compose up -d                 # pgvector on 5432
psql -h localhost -U postgres -c "CREATE EXTENSION IF NOT EXISTS vector;"
nvm use                              # reads frontend/.nvmrc -> 20.19.6
cd frontend && npm ci && ng build     # must pass before Day 3, not on demo morning
```

Ingestion — assert the Day-0 snapshot has ≈3,000+ tenders and that a known detail page (`id=1327991`, RAB-8 Barishal, "Procurement of Motor Vehical Parts") parses with a non-empty *Eligibility of Tenderer*. World Bank: assert `project_ctry_name_exact=Bangladesh` returns strictly fewer than 417,948.

Matching — unit test the BRD's own example: profile "ICT systems implementation" must match a tender titled "capacity building in digital governance" **above** the keyword baseline's score for the same pair. This one test *is* the product thesis.

Eligibility — table-driven tests: turnover below threshold fails; missing certification fails with the certification named; both present passes. Assert no LLM call occurs on this path.

Scheduling — assert every cron parses under `Asia/Dhaka` (not server-local). Run `egpDiscovery` twice against an unchanged portal and assert the second run fetches **zero** detail pages — that proves the 20-known-ID stop rule works and that we are not re-crawling e-GP every 30 minutes. Force a simulated 5xx and assert backoff engages rather than retrying immediately. Start under the `demo` profile and assert no scheduled job fires.

Frontend — verify the **packaged** build, not just `ng serve`: run `./gradlew ngBuild bootRun`, load `http://localhost:8080/`, then **hard-refresh on a deep link** such as `/tenders/1327991`. A 404 there means SPA fallback routing is not configured — the single most common Angular-on-Spring failure, and one that never appears under `ng serve`.

End-to-end — `POST /api/pipeline/run` then load `/` and confirm a ranked shortlist with grades, evidence, and gaps. Then restart under the `cached` profile with networking disabled and confirm the identical shortlist renders — this is the demo-day safety net, and it must be tested cold, not assumed.

Benchmark — `/eval` computes precision@5 for both matchers on the held-out 12. Run **once**.

> Report the number honestly. With 12 tenders and top-5, semantic may beat keyword only 4-to-3 — within noise, and a sharp judge will say so. Lead the demo with the *mechanism* (the vocabulary-gap match above, shown with evidence) and present the metric as supporting, not headline. Overclaiming a thin metric loses more credibility than a modest honest one gains.

Risks

Risk	Mitigation
e-GP blocks us before 12 Sep	Day-0 snapshot — the single most important task on this plan
Selenium flaky on demo day	`CachedFileSource` profile switch, rehearsed cold
Boot 4.1 rejects Java 17	Install JDK 21; discovered on Day 0, not Day 3
No free LLM key obtained	`TemplateSummaryWriter` is built first and ships regardless
Benchmark margin is thin	Framing above — mechanism leads, metric supports
Bangla swap breaks matching	Branch only, Day 3, never merged if not green
Scope creep from 8 extra features	Features 1–3 are core; 4–7 are cuttable in order; 8 is stretch
Angular costs more than server-rendered UI in 4 days	Mitigated by structure, not by hope: API frozen and stubbed Day 1, five screens only, no component library, `shortlist/` and `benchmark/` built before anything else. If Day 3 slips, `profile/` and `pipeline/` degrade to read-only views
Wrong Node version burns an hour	`frontend/.nvmrc` pins 20.19.6; `nvm use` is in the setup commands
Packaged build breaks on demo morning	`ngBuild` wired and deep-link-tested by Day 3 midday